

Chapter 7

Linux Debugging, Tracing, Performance

5 Lecture · 영상 90 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L31 /proc 를 이용한 Process 내부 분석

L32 strace, ltrace, gdb, Core Dump

L33 perf 를 이용한 Performance 분석

L34 ftrace 와 Kernel Function Tracing

L35 eBPF 와 bpftrace

L31

/proc 를 이용한 Process 내부 분석

Chapter 7. Linux Debugging, Tracing, Performance
도구를 설치하기 전에 , 이미 있는 것부터

영상 17 분

▶ 애니메이션 [C7-1 /proc의 동작](#)

이 Chapter 의 지도

" 느리다 / 이상하다 "

└ 1. CPU 를 쓰는가 , 기다리는가 ?	/proc, top	L31
└ 쓴다 (On-CPU) → 어느 함수 ?	perf	L33
└ 기다린다 (Off-CPU) → 무엇을 ?	wchan, strace -c -w	L31, L32
└ 2. Kernel 과 무슨 대화를 ?	strace	L32
└ 3. 죽었다 / 멈췄다	Core Dump, gdb -p	L32
└ 4. Kernel 안에서는 ?	ftrace	L34
└ 5. 운영 중에 , 싸게 , 원하는 것만	eBPF / bpftrace	L35

관찰 대상

```
./target & # Source 를 보지 않았다고 가정한다  
ls /proc/$P
```

```
status stat task/ fd/ fdinfo/ io maps smaps_rollup wchan stack  
syscall limits sched ...
```

읽는 순간 Kernel 이 task_struct, mm_struct, files_struct 를 들여다보고 **문자열로 만들어 준다** (L12)

→ *Terminal 전환*

status: 한 장짜리 요약

항목	읽는 법
State	R / S / D (Disk I/O) / T / Z
Threads	계속 늘면 Thread Leak
VmRSS	계속 늘면 Memory Leak
voluntary_ctxt_switches	많으면 기다리는 프로그램
nonvoluntary_ctxt_switches	많으면 CPU 를 다투는 프로그램
TracerPid	strace/gdb 가 붙어 있는가

task: Thread 별로 보기

```
566 target S    0 0 hrtimer_nanosleep
568 burner S 148 2 hrtimer_nanosleep
569 logger S    0 1 hrtimer_nanosleep
```

← CPU 를 쓰는 것은 이 Thread 뿐

Thread 마다 Directory. 구조는 /proc/PID 와 같다

- **pthread_setname_np** 로 이름을 붙여 두자

stat 의 14, 15 번 = utime, stime (Tick). 마지막) 뒤부터 **Parsing**

fd 와 io

```
3 -> ../target.log    (l-wx)                pos: 468 → 558 (1 초 뒤 )
wchar: 456      syscw: 57      write_bytes: 4096
```

wchar	write 계열로 넘긴 Byte. Page Cache 포함
syscw	횟수 → 456 / 57 = 8 Byte/ 회
write_bytes	실제 Block Device 로 (4KB 단위 , L15)

→ " 너무 작은 I/O" 를 Source 없이 찾았다

Memory 와 " 어디에서 멈췄는가 "

Pss_Anon: 3200 → 4224 → 5248 kB (2 초마다 정확히 1024 kB)

Rss: 공유 Page 통째로 / **Pss**: 공유하는 Process 수로 나눔

```
wchan : hrtimer_nanosleep  
stack : hrtimer_nanosleep ← common_nsleep ← __x64_sys_clock_nanosleep ← do_syscall_64 ← entry_SYSCALL_64  
syscall: 230 ...
```

futex_* = Lock 대기 · ep_poll = epoll_wait · io_schedule = Disk (State D)

대상을 전혀 건드리지 않는다 → 운영 Service 에 가장 먼저

/proc 의 동작

화면 전환 → [animation.html](#)

status 읽기 → top 은 CPU% 를 어떻게 구하는가

▶ 애니메이션 [C7-1 /proc의 동작](#)

procmon: /proc 만으로 만드는 top

$$\text{CPU\%} = ((\text{utime} + \text{stime}) \text{ 증가분} / \text{CLK_TCK}) / \text{경과 시간} \times 100$$

sec	S	CPU%	thr	RSS(kB)	fds	write/s
1	S	30.0	3	2912	4	10
2	S	30.0	3	3940	4	10
4	S	30.0	3	4968	4	10

값은 누적 → 두 Sample 의 차이 · **Final Project 의 Process Monitoring** 부분

→ Code + Terminal

정리

첫 질문 : **CPU** 를 쓰는가 , 기다리는가

/proc/PID = Kernel 자료 구조의 창 . 대상을 건드리지 않는다

- status, task/, fd·io, smaps_rollup(Pss), wchan·stack·syscall
- 누적값 → 두 Sample 의 차이

다음 : L32. syscw 가 57 이라는 것은 알지만 , 그 57 번이 각각 무엇이었는가

L32

strace, ltrace, gdb, Core Dump

System Call 을 하나하나 보고 , 죽은 프로그램을 부검하고 , 멈춘 프로그램을 진찰한다 . 모두 ptrace 위에 있다

영상 19 분

▶ **애니메이션** [C7-2 ptrace: strace와 gdb의 동작 원리](#)

느린데 CPU 는 놀고 있다 : strace -c

```
$ strace -c ./chatty slow
100.00    0.199107    0    200000    write
$ strace -c ./chatty fast
0.00    0.000000    0    657    write
```

200,000 번대 657 번 . sys 시간이 user 보다 크다면 strace

seconds = System Call 안의 **CPU** 시간 . 기다린 시간까지 보려면 **-w**

→ Terminal 전환

자주 쓰는 Option

<code>-e trace=write,openat,%file,%network</code>	범위를 좁힌다
<code>-T, -tt, -r</code>	소요 시간 , 시각 , 간격
<code>-y</code>	fd → 경로
<code>-f</code>	Child 와 Thread 도 . Multi-Thread 에는 필수
<code>-p PID</code>	실행 중인 Process 에 붙는다
<code>-k</code>	호출한 User Stack
<code>-Z</code>	실패한 호출만

불친절한 오류의 원인 찾기

```
$ ./chatty missing
```

오류 : 설정을 불러올 수 없습니다

```
$ strace -e trace=openat ./chatty missing
```

```
openat(..., "./chatty.conf", O_RDONLY)      = -1 ENOENT
```

```
openat(..., "/etc/chatty/chatty.conf", O_RDONLY) = -1 ENOENT
```

```
openat(..., "/usr/local/etc/chatty.conf", O_RDONLY) = -1 ENOENT
```

strace 의 가장 흔하고 가장 효과적인 용도

→ *Terminal 전환*

오류 주입

```
$ strace -e trace=write -e inject=write:error=ENOSPC:when=3 ./chatty slow  
write(3, "record 2\n", 9) = -1 ENOSPC (No space left on device) (INJECTED)  
+++ exited with 1 +++
```

Disk 를 채우지 않고 "Disk Full" 시험 . delay_enter=, signal= 도 가능

오류 처리 경로는 평소에 실행되지 않는다 → 강제로 실행해 봐야 검증된다

ptrace 와 그 비용

Tracee: write() → Kernel 진입 → 멈춤 → [strace 가 Register 와 인자를 읽음] → 계속
→ 실제 수행 → 멈춤 → [strace 가 return 값을 읽음] → 계속

System Call 한 번에 **Context Switch 4 번**

```
./chatty slow                0.073 s
strace -c ./chatty slow      1.339 s    ← 18 배
ltrace -c ./chatty fast     12.8 s      ← 900 배
```

운영 Service 에 strace -p 는 신중하게 . Tracer 는 하나만 붙을 수 있다 (TracerPid)

Core Dump: 죽은 프로그램 부검

```
ulimit -c unlimited
sudo sysctl -w kernel.core_pattern=core.%e.%p      # Ubuntu 기본은 apport 로 Pipe
./crash                                              # Segmentation fault (core dumped)
gdb ./crash core.crash.*
```

Core = 죽는 순간의 **Memory + Register**. 나중에, 다른 장비에서, 몇 번이고

gdb 로 원인 찾기

```
#0  name_length (u=0x0) at crash.c:25      ← 어디서 : NULL 역참조
#1  report (tbl=..., n=3, id=7) at crash.c:32 ← 어떻게 : id=7
#2  main () at crash.c:41
(gdb) frame 1 → print u                    $1 = (struct user *) 0x0
(gdb) frame 2 → info locals                 ids = {1, 3, 7, 2}, i = 2      ← 왜 : Table 에 7 이 없다
```

필요한 것 : **같은 Binary + Debug 정보 (-g)**. Core 에는 비밀 정보가 들어 있을 수 있다

→ *Terminal 전환*

`gdb -p`: 멈춘 프로그램 진찰

```
wchan: 세 Thread 모두 futex_do_wait          ← Lock 대기 (L31 의 방법 )
(gdb) thread apply all bt
    "transfer" (LWP 679): pthread_mutex_lock(<m_log>)
    "audit"    (LWP 680): pthread_mutex_lock(<m_account>)
(gdb) print m_account.__data.__owner          $1 = 679
(gdb) print m_log.__data.__owner              $2 = 680
```

서로가 가진 것을 기다린다 = **Deadlock**

`detach` · **`gcore`**(죽이지 않고 Core) · `gdb -batch -p PID -ex "thread apply all bt"`

→ Terminal 전환

ptrace의 동작

화면 전환 → [animation.html](#)

평소의 System Call → strace가 붙었을 때 → gdb의 Breakpoint

▶ **애니메이션** [C7-2 ptrace: strace와 gdb의 동작 원리](#)

정리

strace -c(-w), -e trace=%file, -f, -p, inject

- ptrace: System Call 마다 Context Switch 4 번 . 18 배 , 900 배

Core Dump: ulimit -c, core_pattern, bt → frame → print

gdb -p: thread apply all bt, __owner, gcore

다음 : L33. System Call 을 하지 않고 CPU 를 100% 쓰는 프로그램

L33

perf 를 이용한 Performance 분석

CPU 에게 일정 간격으로 묻는다 . " 지금 어느 함수였나 ?"

영상 18 분

▶ 애니메이션 [C7-3 perf Sampling](#)

준비

```
sudo apt install -y linux-tools-common linux-tools-generic linux-tools-$(uname -r)
cat /proc/sys/kernel/perf_event_paranoid          # Ubuntu 기본 4 → sudo 필요
```

```
CFLAGS := -O2 -g -fno-omit-frame-pointer
```

-O2: 실제 배포와 같은 최적화로 측정한다

-g: 주소 → 함수, 줄

-fno-omit-frame-pointer: 호출 경로를 거슬러 올라가려면 필요

perf stat: 세기

```
1083.88 msec task-clock          #    1.002 CPUs utilized    ← On-CPU. perf 의 대상
      23      context-switches
      71      page-faults
<not supported> cycles, instructions    ← VirtualBox
1.08 s elapsed    1.08 s user    0.00 s sys    ← strace 로는 볼 것이 없다
```

perf record / report

```
perf record -g ./hot
perf report --stdio --no-children -g none
```

```
65.78%  hot  hot      [.] checksum
23.84%  hot  hot      [.] normalize
 3.10%  hot  libc.so.6 [.] __printf_fp_buffer_1.isra.0
 2.12%  hot  libc.so.6 [.] __printf_buffer
```

최적화할 곳은 checksum. 실행 중인 Process 에도 : `perf record -g -p PID --sleep 10`

→ *Terminal 전환*

Sampling 의 원리

화면 전환 → [animation.html](#)

On-CPU Sampling → Sample 이 적으면 → 잠자는 프로그램

▶ 애니메이션 [C7-3 perf Sampling](#)

Sampling 의 성질

통계이다	수 초 이상 실행 . 짧은 함수는 안 보일 수 있다
Overhead 가 작다	1~5%. strace 의 수십 배와 비교 . 운영 환경 가능
호출 횟수는 모른다	그것은 Tracing 의 영역 (L35 uprobe)
Source 수정 불필요	

Self 와 Children, Call Graph

Children	Self	
99.92%	0.02%	main
65.89%	65.78%	checksum
23.84%	23.84%	normalize
10.09%	0.11%	format
9.95%	0.39%	__snprintf_chk

← 자기 코드는 가볍지만 snprintf 를 불러 10%

Self: 자기 코드 → " 어느 함수의 코드가 뜨거운가 "

Children: Stack 에 있던 모든 Sample → " 어느 기능이 비싼가 "

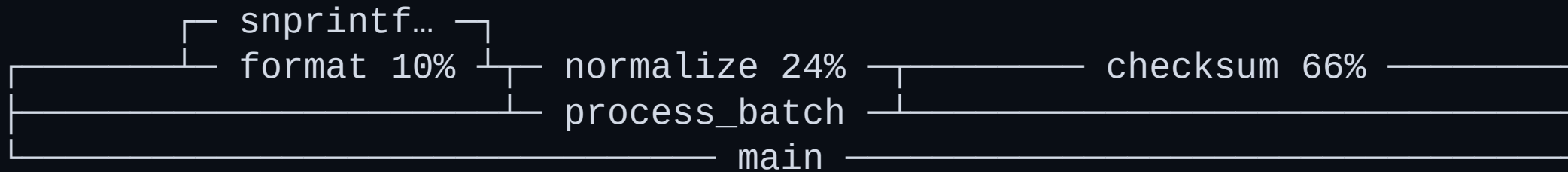
Library 함수가 뜨겁다면 누가 불렀는지를 봐야 고칠 수 있다 → -g

→ Terminal 전환

annotate, Flame Graph

```
36.49 :    xor    %rcx,%rax
36.18 :    cmp    %rdx,%rdi
21.30 :    imul   %rsi,%rax
```

← Skid: "이 근처" 로 읽는다



가로 폭 = 시간 · 윗면이 넓은 상자 = 최적화 대상 · 가로는 시간 순이 아니다

Hardware Counter: 왜 느린가

행 (row) 우선 : 8.6 ms 열 (col) 우선 : 143.6 ms ← 17 배 . 명령어 수는 같다

(Native) instructions 같음 IPC 3.1 → 0.40 cache-misses 1% → 93%

CPU 가 **Memory** 를 기다리며 놀고 있었다

VirtualBox: <not supported>. Sampling 은 cpu-clock 으로 정상 동작

→ Terminal 전환

perf 가 보지 못하는 것 : Off-CPU

```
real 2.10 s    user 0.03 s    sys 0.01 s          41.60 msec task-clock # 0.020 CPUs utilized
perf report:  65.78% compute      ← 0.04 초의 66%
```

```
$ strace -c -w ./offcpu
97.90      2.015514      100      clock_nanosleep      ← 진짜 원인
2.01      0.041411      100      fsync
```

Sampling 은 CPU 위에 있을 때만 찍힌다

CPUs utilized ≈ 1	perf record
$\ll 1$	strace -c -w, wchan, eBPF offcputime

→ Terminal 전환

정리

perf stat: 센다 . CPUs utilized 부터

perf record -g → report: Sample 비율 = 시간 비율 . 1~5%

Self vs Children, -fno-omit-frame-pointer, -O2

- Flame Graph: 폭 = 시간 , 가로는 시간 순 아님
- HW Counter = " 왜 ". VirtualBox 미지원
- **Off-CPU** 는 보이지 않는다

다음 : L34. " 언제 , 어떤 순서로 "

L34

ftrace와 Kernel Function Tracing

Kernel에 내장된 Tracer. echo하고 cat하면 된다

영상 17분

▶ 애니메이션 [C7-4 ftrace의 동작 원리](#)

Sampling 과 Tracing

	Sampling (perf)	Tracing (ftrace, strace, eBPF)
방법	일정 간격으로 " 지금 어디 ?"	사건마다 기록
알 수 있는 것	시간의 분포	순서 , 횟수 , 간격 , 인자
Overhead	일정	사건의 빈도에 비례
질문	어디가 뜨거운가	무슨 일이 , 어떤 순서로

strace = 경계만 . ftrace = 경계 안쪽

tracefs

```
/sys/kernel/tracing/  
tracing_on    trace    trace_pipe    current_tracer    available_tracers  
events/<subsystem>/<event>/{enable,format}  
set_event_pid    set_ftrace_pid    set_ftrace_filter    set_graph_function  
trace_marker    buffer_size_kb
```

Tracepoint = Kernel 개발자가 미리 심어 둔 고정 관측 지점 (약 1,500 개). 이름과 인자가 안정적

→ *Terminal 전환*

한 Process 만 , 내 표식과 함께

```
sh -c "echo \$\$ > set_event_pid; echo 1 > tracing_on; exec \"\${@}\" sh \"${@}\""
```

자기 PID 를 Filter 에 쓰고 → 기록을 켜고 → **exec**(PID 유지) 로 대상이 된다

```
mfd = open("/sys/kernel/tracing/trace_marker", O_WRONLY);  
mark("phase 2: fsync 시작");    fsync(fd);    mark("phase 3: sleep 시작");
```

내 코드의 구간 경계가 Kernel Event 와 같은 **Timeline** 에

→ Code 화면

trace 읽기 : fsync 안에서 일어난 일

```

marker-869 ...713655: tracing_mark_write: phase 1: write 시작
marker-869 ...713687: sys_write(fd: 4, count: 0x1000)    × 4           45 µs, sched_switch 없음
marker-869 ...713700: tracing_mark_write: phase 2: fsync 시작
marker-869 ...713700: sys_fsync(fd: 4)
marker-869 ...713757: sched_switch: prev_comm=marker prev_state=D ==> swapper ← Disk 대기
<...>-12 ...713898: sched_switch: prev_comm=kworker/u4:0 ==> next_comm=marker
marker-869 ...714412: sys_fsync -> 0x0                      712 µs
marker-869 ...714419: sys_clock_nanosleep(...)
marker-869 ...714420: sched_switch: prev_comm=marker prev_state=S ==> swapper ← Sleep
<idle>-0 ...724525: sched_switch: ... ==> next_comm=marker      10.1 ms 뒤

```

→ Terminal 전환

function Tracer

```
echo 'vfs_read vfs_write' > set_ftrace_filter      # 반드시 먼저  
echo function > current_tracer
```

```
cat-2901  [001] ... 412.118201: vfs_read <-ksys_read  
cat-2901  [001] ... 412.118302: vfs_write <-ksys_write
```

Kernel 함수 약 5 만 개 . Wildcard: 'ext4_*

Filter 없이 켜면 초당 수백만 건 → System 이 느려지고 Buffer 는 1 초도 못 버틴다

function_graph Tracer

```

1)          | vfs_read() {
1)  0.902 us |   rw_verify_area();
1)          |   ext4_file_read_iter() {
1)          |       filemap_read() {
1)  0.851 us |           filemap_get_pages();
1)  0.702 us |           copy_page_to_iter();
1)  3.104 us |       }
1)  3.927 us |   }
1)  6.210 us | }
```

L11, L12의 그림이 **실제 함수 이름**으로 . Cache Hit → Block Layer 함수 없음

tracing_thresh: 오래 걸린 함수만 · + 100μs, ! 1ms, # 10ms

→ Terminal 전환

ftrace 는 어떻게 함수를 가로채는가

화면 전환 → [animation.html](#)

꺼져 있을 때 (nop) → function Tracer 를 켜면 (call 로 Patch) → Tracepoint

▶ 애니메이션 [C7-4 ftrace의 동작 원리](#)

trace-cmd, 주의점

```
sudo trace-cmd record -e syscalls:sys_enter_fsync -e sched:sched_switch ./marker
sudo trace-cmd record -p function_graph -g vfs_read cat /etc/hostname
sudo trace-cmd report
```

뒷정리 : 끝 때까지 켜져 있다

tracefs 설정은 System 에 하나 → instances/ 로 분리

- Ring Buffer 가 넘치면 오래된 것부터 덮어쓴다

root, Lockdown. sudo echo 1 > file 은 실패한다 → | sudo tee

정리

- Tracing = 순서, 횟수, 간격
tracefs. Tracepoint Event + set_event_pid
trace_marker: 내 구간을 같은 Timeline 에
write 는 안 자고, fsync 는 **D**, nanosleep 은 **S**
function / function_graph. **Filter** 먼저
- 평소 Overhead 0: nop ↔ call

다음 : L35. 기록을 읽는 대신 Kernel 안에서 집계까지

L35

eBPF 와 bpftrace

검증된 작은 프로그램을 Kernel 안에서 실행한다 . 필요한 것만 , 그 자리에서 집계하고 , 결과만 가져온다

영상 19 분

▶ 애니메이션 [C7-5 eBPF의 동작](#)

왜 또 다른 도구인가

chatty slow 의 write 20 만 번 . 알고 싶은 것은 숫자 몇 개

	사건마다 하는 일	집계하는 곳
strace	Process 정지 → Context Switch × 4	User Space → 18 배
ftrace	Ring Buffer 에 기록	사람이 읽음
eBPF	Kernel 안에서 작은 함수 실행 (수십 ~ 수백 ns)	Kernel 안 (Map)

eBPF 의 구조

```
bpftrace Script → ①Compile → Bytecode → ②bpf() →  
  ③ Verifier : 반드시 종료? Memory 접근 안전? → 통과 못 하면 Load 거부  
  ④ JIT      : Native 기계어  
  ⑤ Attach   : tracepoint / kprobe / uprobe / perf event  
Event → eBPF Program 실행 → ⑥ BPF Map  
                               ↳ ⑦ → bpftrace 가 읽어 출력
```

Kernel Module 과의 결정적 차이 : Module 은 Bug 하나로 Kernel 을 죽인다 . eBPF 는
검증을 통과해야만 실행된다

Event 에서 Map 까지

화면 전환 → [animation.html](#)

Load → Event 처리 → Verifier 의 거부

▶ 애니메이션 [C7-5 eBPF의 동작](#)

bpftrace 문법

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); } interval:s:3 { exit(); }'
```

```
probe /filter/ { action }
```

내장 변수	pid tid comm nsecs args retval probe cpid
Map @name[key]	Kernel 안 . 종료 시 자동 출력
집계	count() sum() avg() hist()
찾기	bpftrace -l 'tracepoint:syscalls:*open*', -lv

→ *Terminal 전환*

opensnoop: 누가 어떤 File 을 여는가

```
tracepoint:syscalls:sys_enter_openat { @path[tid] = args->filename; }
tracepoint:syscalls:sys_exit_openat /@path[tid]/ {
    printf("%-8d %-16s %4d %s\n", pid, comm, args->ret, str(@path[tid])); delete(@path[tid]);
}
```

```
926  chatty    -2  ./chatty.conf
926  chatty    -2  /etc/chatty/chatty.conf
```

진입 (인자) 과 복귀 (결과) 를 **@map[tid]** 로 잇는다 = 가장 자주 쓰는 Pattern

-c 없이 실행하면 **System 전체** → " 누가 이 File 을 건드리는지 모르겠다 "

→ *Code + Terminal*

Histogram

```
@size_bytes:
[8, 16)          200000 | @@@@ |
@latency_ns:
[256, 512)       196908 | @@@@ |
[512, 1K)         2116  |
...
[16K, 32K)        22   |
```

- write 20 만 번이 전부 8~16Byte

꼬리 (Tail Latency): 평균에는 묻힌다

20 만 건 중 User Space 로 넘어온 것은 이 표뿐 . strace 18 배 vs eBPF **Event 당 1 μ s 미만**

→ Terminal 전환

uprobe: 내 프로그램의 함수

```
@calls[...checksum]: 2000      @avg_us[...checksum]: 70  
@calls[...normalize]: 2000     @avg_us[...normalize]: 44  
@calls[...format]: 2000        @avg_us[...format]: 8
```

Source 수정 없이, 재 Compile 없이 . perf 가 모르는 **호출 횟수와 호출당 시간**

단, 호출마다 Trap → **호출당 수 μ s**. 짧은 함수는 부풀려진다

자주 불리는 짧은 함수 = perf / 횟수와 분포 = uprobe

→ *Terminal 전환*

Probe 의 종류 , Verifier

Probe	
tracepoint	안정적 , 인자에 이름 . 먼저 찾을 것
kprobe / kretprobe, fentry	임의의 Kernel 함수 . Version 의존
uprobe / uretprobe, usdt	User 함수 , Library
profile:hz:99	Sampling (Flame Graph 재료)
interval, BEGIN, END	

Verifier: 반드시 종료 · 범위 안의 Memory 접근 · NULL 검사 · Helper 로만 Kernel Memory 읽기 · Stack 512B

bpftrace(조사) / BCC(도구 100 여 개) / libbpf + CO-RE(제품)

Chapter 7 정리 : 도구 선택

질문	도구
CPU 를 쓰는가 , 기다리는가 ?	/proc, top
어디에서 멈췄는가 ?	wchan, stack
Kernel 과 무슨 대화를 ? 왜 실패 ?	strace
왜 죽었나 / 멈췄나 ?	Core + gdb / gdb -p
어느 함수가 CPU 를 ?	perf record -g
기다리는 시간은 ?	strace -c -w, offcputime
Kernel 안에서 어떤 순서로 ?	ftrace
운영 중에 , 싸게 , 집계해서 ?	bpftrace

정리

- eBPF = Verifier → JIT → Attach → Map. Kernel 안에서 집계
- bpftrace: probe /filter/ { action }, @map, hist(), @start[tid] Pattern
- tracepoint 먼저 , kprobe, uprobe
 - 운영 환경에 쓸 수 있는 비용

Worksheet 20 문제 → Chapter 8: Capability, seccomp, Namespace, Cgroup, Final Project

다음 시간 · Chapter 8

Linux Isolation, Security 그리고 Final Project

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog